MDI3003

ADVANCED PREDICTIVE ANALYTICS

Dr. Durgesh Kumar

Assistant Professor (Senior), SCOPE, VIT Vellore

STUDENT LABORATORY INSTRUCTION MANUAL

Lab 01

House Price Prediction using Linear Regression

From statistical baseline to an end-to-end, reproducible regression study

Core laboratory duration	3 hours
Extended study	One week / instructor-defined submission window
Primary method	Simple and Multiple Linear Regression
Comparative methods	Regularized, polynomial, tree and ensemble regression
Recommended environment	Python 3.10+ in Jupyter Notebook or Google Colab

Prepared as a rigorous, student-facing laboratory guide with industry-style documentation requirements

How to Use This Manual

Two-tier scope design

The formal laboratory slot is treated as a focused 3-hour core exercise in linear regression. The requirement to use multiple datasets, compare at least five models, perform tuning, and write an industry-style report is structured as an extended study. This avoids reducing the core lab to rushed code execution and preserves time for reasoning, diagnostics, and interpretation.

Students should complete the pre-lab preparation before entering the laboratory. During the scheduled session, they must finish the core workflow and obtain a defensible linear-regression baseline. The comparative study may then be completed individually or in teams, according to the instructor's policy.

Contents

1. Lab Title and Positioning

Lab title: House Price Prediction using Linear Regression

This laboratory introduces regression as an end-to-end predictive analytics problem. Linear Regression is the scientific baseline and interpretability anchor. More flexible regression models are included as controlled comparisons, not as substitutes for understanding the baseline.

Central question

How accurately, reliably, and transparently can property value be predicted from measurable property and location characteristics?

2. Lab Objectives

- Translate a real-estate valuation need into a supervised regression problem with a clearly defined target and success criteria.
- Develop simple and multiple Linear Regression models and interpret coefficients in the context of house pricing.
- Build a leakage-safe preprocessing and modeling pipeline for numerical and categorical data.
- Evaluate predictive performance using error metrics, cross-validation, residual diagnostics, and train-test comparison.
- Compare Linear Regression with regularized, nonlinear, tree-based, and ensemble regressors.
- Document assumptions, decisions, limitations, and model risks using an industry-style technical report.

3. Learning Outcomes

After completing the laboratory and extended study, a student should be able to:

Outcome	Demonstrable capability
LO1	distinguish prediction from explanation and identify when regression is appropriate;
LO2	define features, target, observation unit, prediction horizon, and evaluation protocol;
LO3	audit a tabular dataset for data types, missing values, duplicates, outliers, skewness, and leakage;
LO4	implement reproducible preprocessing with imputation, encoding, scaling, and pipelines;
LO5	derive and explain the ordinary least-squares objective and major regression metrics;
LO6	train, validate, tune, and compare at least five regression models;
LO7	diagnose underfitting, overfitting, heteroscedasticity, influential observations, and multicollinearity;
LO8	communicate results in business units, justify the selected model, and state deployment limitations.

4. Problem Statement

A real-estate analytics team needs a repeatable method for estimating the market value of a residential property from available attributes such as living area, age, condition, neighborhood, accessibility, and nearby amenities. The objective is to learn a function that maps property attributes to a continuous price or price-per-unit-area target.

$$y = f(x_1, x_2, \ldots, x_p) + \varepsilon \quad \text{[general regression model]}$$

The model must be evaluated on observations not used during fitting. A technically accurate model is not sufficient by itself: students must also establish reproducibility, interpretability, error behavior, and limitations.

4.1 Required analytical questions

- Which property attributes are most strongly associated with price?
- How strong is a single-feature Linear Regression baseline?
- How much does a properly preprocessed multiple-feature model improve performance?
- Do polynomial terms or regularization improve generalization?
- When do tree and ensemble methods outperform the linear baseline, and what interpretability is lost?
- Are model errors systematically larger for expensive, old, small, or geographically distinct properties?

5. Real-World Motivation and Industry Relevance

Stakeholder	Typical use	Why prediction quality matters
Property buyers and sellers	Price discovery and negotiation support	Large errors can distort affordability decisions.
Banks and mortgage firms	Valuation checks and collateral risk	Systematic overvaluation increases credit risk.
Developers and investors	Site selection, portfolio screening, return estimation	Model bias can misallocate capital.
Tax and public agencies	Mass appraisal and anomaly detection	Transparency and consistency are essential.
Real-estate platforms	Automated valuation estimates and ranking	Predictions must remain current and location-aware.

Professional practice

A model is a decision-support instrument, not an unquestionable price authority. Predictions should be accompanied by error ranges, data coverage statements, and escalation rules for unusual properties.

6. Scope, Schedule, and Deliverables

6.1 Pre-lab preparation

- Read the sections on supervised learning, Linear Regression, train-test split, and regression metrics.
- Create the Python environment and verify that the required libraries import successfully.
- Obtain the assigned dataset and read its data dictionary before the laboratory.
- Prepare a notebook with title, student details, problem statement, and section headings.

6.2 Suggested 3-hour core laboratory plan

Time	Activity	Required checkpoint
0-20 min	Problem framing, dataset loading, schema audit	Target and observation unit correctly identified
20-55 min	Data quality checks and focused EDA	At least four defensible observations
55-90 min	Train-test split and leakage-safe preprocessing	Reusable preprocessing pipeline
90-125 min	Simple and multiple Linear Regression	Baseline and multivariate results
125-155 min	Metrics, residual plots, assumption checks	MAE, RMSE, R^2 and residual interpretation
155-180 min	Summary, limitations and submission packaging	Core notebook executes end-to-end

6.3 Extended comparative study

- Use at least two datasets, including one dataset with mixed numerical and categorical variables.
- Compare at least five regression models using the same split and evaluation protocol.
- Tune at least one regularized or ensemble model using cross-validation.
- Prepare an 8-12 page industry-style report, excluding appendices and code listings.

6.4 Deliverables

Deliverable	Required content
Executable notebook	Complete workflow, comments, fixed seed, outputs, plots and result tables.
Technical report	Business framing, methodology, results, interpretation, limitations and conclusion.
README or run note	Environment, dataset acquisition, execution order and file structure.
Optional model artifact	Serialized final pipeline using joblib, with clear version note.

7. Required Software and Python Libraries

Category	Recommended tool/library	Purpose
Environment	Jupyter Notebook / Google Colab / VS Code	Interactive development and reproducible execution
Core data	NumPy, pandas	Numerical operations and tabular data management
Visualization	Matplotlib, Seaborn	EDA and diagnostic plots
Machine learning	scikit-learn	Pipelines, regression models, validation and metrics
Statistical diagnostics	statsmodels	VIF, influence measures and inferential diagnostics
Optional boosting	XGBoost	Advanced gradient-boosted tree comparison
Persistence	joblib	Save and reload the fitted pipeline

Environment setup

```
# Core installation for Colab or a clean environment
!pip install -q pandas numpy matplotlib seaborn scikit-learn statsmodels ucimlrepo joblib

# Optional extension
!pip install -q xgboost
```

Reproducibility rule

Record Python and package versions. Use a fixed random seed for splitting and stochastic models. Do not change the test set after viewing its results.

8. Recommended Real-World Datasets

The datasets below form a progression from a small, clean valuation problem to a realistic, high-dimensional mixed-type problem. Students should use at least two.

Dataset	Scale and target	Strengths	Recommended role
UCI Real Estate Valuation	414 instances; 6 predictors; target is house price per unit area	Small, clean, no missing values; includes age, transit distance, amenities and coordinates	Core lab or first baseline
California Housing	20,640 samples; 8 numeric predictors; target is median district house value	Direct loading through scikit-learn; useful for scaling, geography and nonlinear patterns	Second dataset or large-data extension
Ames Housing	2,930 residential sales; rich numerical, nominal, ordinal and discrete attributes; sale-price target	Authentic preprocessing, feature engineering, outliers, categorical variables and model selection	Primary extended project
Kaggle House Prices competition	Ames-derived competition with 79 explanatory variables and SalePrice target	Standard train/test competition format and strong benchmarking ecosystem	Optional challenge and leaderboard-style extension

8.1 Dataset-selection guidance

Learning goal	Recommended dataset
Understand regression mechanics and diagnostics	UCI Real Estate Valuation
Study scale, numeric features and spatial effects	California Housing
Practice full preprocessing and realistic modeling	Ames Housing / Kaggle House Prices
Compare classical and ensemble models	Ames Housing or California Housing

8.2 Minimum dataset documentation

- Source, license or terms of use, download date, and citation.
- Observation unit and geographical/time coverage.
- Target definition, unit, and whether it is sale price, median value, or price per unit area.
- Number of rows and columns after loading and after cleaning.
- Data dictionary with variable type and business meaning.
- Known limitations, such as dated prices, geographic specificity, aggregation, or target censoring.

Avoid a common conceptual error

California Housing predicts the median value of a census block group rather than the sale price of an individual house. UCI predicts price per unit area. These targets must be described accurately in the report.

9. Theoretical Background

9.1 Supervised learning and regression

In supervised learning, the training data contain input-output pairs. Regression is used when the output is continuous. The fitted model estimates the conditional relationship between property characteristics and price.

$$D = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n} \quad \text{[training dataset]}$$

$$\hat{y}^{(i)} = f\theta(x^{(i)}) \quad \text{[prediction]}$$

9.2 Simple Linear Regression

Simple Linear Regression uses one predictor. It is valuable as an interpretable baseline and for visualizing the fitted relationship.

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \quad \text{[simple linear model]}$$

β_0 is the intercept; β_1 is the expected change in the target associated with a one-unit increase in x, holding no other variable because only one predictor is present. ε represents unexplained variation.

9.3 Multiple Linear Regression

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} + \varepsilon_i \quad \text{[multiple linear model]}$$

$$y = X\beta + \varepsilon \quad \text{[matrix form]}$$

For a correctly specified model, each coefficient describes a conditional association: the expected change in predicted price for a one-unit change in that feature while the other included features are held constant. This is not automatically a causal effect.

9.4 Ordinary Least Squares and the cost function

$$\mathrm{SSE}(\beta) = \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad \text{[sum of squared errors]}$$

$$J(\beta) = (1 / 2n) \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad \text{[mean squared cost]}$$

Ordinary Least Squares selects coefficients that minimize the squared residuals. When the required matrix inverse exists, the closed-form solution is:

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad \text{[normal equation]}$$

In software, numerically stable matrix factorization is used rather than explicitly computing the inverse.

9.5 Gradient descent: conceptual view

$$\beta_j \leftarrow \beta_j - \alpha \, (\partial J / \partial \beta_j) \quad \text{[parameter update]}$$

$$\partial J / \partial \beta_j = -(1/n) \Sigma_{i=1}^{n} x_{ij}(y_i - \hat{y}_i) \quad \text{[gradient component]}$$

The learning rate α controls the step size. Too large a value may diverge; too small a value may converge slowly. scikit-learn LinearRegression normally uses a direct least-squares solver, but gradient descent remains important for large-scale and deep-learning models.

9.6 Polynomial Regression

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta\_d x^d \quad \text{[degree-d polynomial]}$$

Polynomial Regression is linear in the coefficients but nonlinear in the original predictor. Degree must be controlled because higher degrees can fit noise and create unstable extrapolation.

9.7 Regularized linear models

$$\text{Ridge: minimize } \mathrm{SSE} + \lambda \Sigma_{j=1}^{p} \beta_j^2 \quad \text{[L2 regularization]}$$

$$\text{Lasso: minimize } \mathrm{SSE} + \lambda \Sigma_{j=1}^{p} |\beta_j| \quad \text{[L1 regularization]}$$

$$\text{Elastic Net: minimize } \mathrm{SSE} + \lambda[\rho \Sigma |\beta_j| + (1-\rho) \Sigma \beta_j^2] \quad \text{[combined penalty]}$$

Regularization trades a small amount of training fit for improved stability and generalization. Scaling numerical features is important because the penalty depends on coefficient magnitude.

9.8 Linear-model assumptions and diagnostics

Assumption / concern	Meaning	Diagnostic evidence
Linearity	Expected target is approximately linear in the transformed predictors	Residual vs fitted plots; component relationships
Independent errors	Residuals are not systematically dependent	Sampling design; time/geographic grouping checks
Homoscedasticity	Residual variance is approximately constant	Residual spread across fitted values
Residual normality	Mainly required for classical inference, not basic prediction	Q-Q plot and residual histogram

Assumption / concern	Meaning	Diagnostic evidence
Low multicollinearity	Predictors are not near-linear combinations	Correlation review and variance inflation factor
No dominant influential points	A few observations do not determine the fit	Leverage, Cook's distance, domain review
Representative data	Training distribution matches intended use	Coverage analysis and train-test distribution comparison

Important distinction

Violation of residual normality does not automatically make predictions unusable. It affects inferential procedures more directly. Prediction quality should be judged using held-out performance and error diagnostics.

10. End-to-End Machine-Learning Workflow

Stage	Required action
1. Business understanding	Define user, decision, target, prediction unit, cost of errors, and success threshold.
2. Data understanding	Audit source, schema, data types, target units, coverage and potential leakage.
3. Holdout design	Reserve a test set before learned preprocessing. Use grouped or temporal splitting when random splitting is unrealistic.
4. Data preparation	Impute, encode, transform, scale, and engineer features inside reusable pipelines.
5. Exploratory analysis	Understand distributions, relationships, outliers, geography and target skew without optimizing on the test set.
6. Baseline modeling	Fit simple and multiple Linear Regression and establish transparent benchmark performance.
7. Validation and tuning	Use cross-validation on training data to compare models and select hyperparameters.
8. Final evaluation	Evaluate the selected configuration once on the untouched test set.
9. Interpretation and risk	Explain drivers, residual patterns, uncertainty, limitations and invalid-use cases.
10. Reproducibility	Package code, versions, seeds, data instructions, tables and report.

10.1 Success criteria

- Technical: lower cross-validated and test RMSE/MAE than a naive mean-prediction baseline.
- Generalization: acceptable gap between training and validation/test performance.
- Diagnostic: no unexplained severe residual pattern or data leakage.
- Business: errors expressed in meaningful currency or target units and discussed by price segment.
- Operational: reproducible pipeline that accepts raw schema and returns predictions.

11. Detailed Step-by-Step Student Instructions

Step 1 - Create a reproducible notebook structure

- Title and student information
- Problem framing and success criteria
- Environment and imports
- Dataset documentation
- Data audit and cleaning
- EDA
- Preprocessing and split
- Baseline models
- Comparative models and tuning
- Evaluation and diagnostics
- Interpretation, limitations and conclusion

Record the execution environment

```
import platform
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

SEED = 42
np.random.seed(SEED)

print('Python:', platform.python_version())
print('pandas:', pd.__version__)
print('scikit-learn:', sklearn.__version__)
```

Step 2 - Load one of the recommended datasets

California Housing loader

```
# Option A: California Housing
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame.rename(columns={'MedHouseVal': 'Price'})
print(housing.DESCR[:1000])
display(df.head())
```

UCI loader

```
# Option B: UCI Real Estate Valuation
from ucimlrepo import fetch_ucirepo

uci = fetch_ucirepo(id=477)
X_uci = uci.data.features.copy()
y_uci = uci.data.targets.copy()
df = pd.concat([X_uci, y_uci], axis=1)
display(df.head())
print(uci.metadata)
```

Ames/Kaggle loader

```
# Option C: Ames / Kaggle House Prices
# Download the CSV using the official course instructions and keep raw data unchanged.
df = pd.read_csv('train.csv')
display(df.head())
print(df.shape)
```

Step 3 - Conduct a data and target audit

Minimum data-quality audit

```
target = 'SalePrice'   # replace for the selected dataset

print('Shape:', df.shape)
print('Target dtype:', df[target].dtype)
display(df.head())
display(df.sample(min(5, len(df)), random_state=SEED))

audit = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'missing_n': df.isna().sum(),
    'missing_pct': 100 * df.isna().mean(),
    'unique_n': df.nunique(dropna=False)
}).sort_values('missing_pct', ascending=False)

display(audit.head(20))
print('Duplicate rows:', df.duplicated().sum())
print(df[target].describe())
```

Do not clean blindly

A missing category such as “no garage” or “no basement” may represent meaningful absence rather than unknown data. Read the data dictionary before selecting an imputation strategy.

Step 4 - Define features and create the holdout split

Remove identifiers that do not generalize unless they encode a defensible grouping. Split before fitting imputers, scalers, encoders, feature selectors, or target-informed transformations.

Random holdout split

```
from sklearn.model_selection import train_test_split

DROP_COLS = ['Id'] if 'Id' in df.columns else []
X = df.drop(columns=[target] + DROP_COLS)
y = df[target].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=SEED
)

print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
```

When random splitting is inappropriate

If repeated properties, neighborhoods, cities, or time periods could appear in both partitions, use GroupShuffleSplit, GroupKFold, or a chronological split. The split must imitate the intended deployment situation.

Step 5 - Perform focused exploratory data analysis

Perform EDA mainly on the training data. The test set is not a source for feature engineering decisions or model selection.

Core EDA

```
train_df = X_train.copy()
train_df[target] = y_train

# Target distribution
fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(train_df[target], kde=True, ax=ax)
ax.set_title('Target Distribution')
plt.show()

# Numerical correlations
num_df = train_df.select_dtypes(include=np.number)
fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(num_df.corr(), cmap='coolwarm', center=0, ax=ax)
ax.set_title('Numerical Correlation Matrix')
plt.show()
```

Required EDA observations should distinguish evidence from speculation. Examples: target skew; nonlinear relationship between living area and price; neighborhood price differences; suspicious extreme values; or location clusters.

Step 6 - Build a leakage-safe preprocessing pipeline

Reusable preprocessing

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler


numeric_features = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_features = X_train.select_dtypes(exclude=np.number).columns.tolist()


numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])


categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])


preprocess = ColumnTransformer([
    ('num', numeric_pipe, numeric_features),
    ('cat', categorical_pipe, categorical_features)
], remainder='drop')
```

For tree models, scaling is not required, but retaining a common preprocessing design makes comparisons easier. Advanced students may create a separate tree-specific pipeline and justify the difference.

Step 7 - Establish a naive baseline

Mean-prediction baseline

```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


naive = DummyRegressor(strategy='mean')
naive.fit(X_train[numeric_features], y_train)
naive_pred = naive.predict(X_test[numeric_features])


print('Naive MAE:', mean_absolute_error(y_test, naive_pred))
print('Naive RMSE:', np.sqrt(mean_squared_error(y_test, naive_pred)))
```

Step 8 - Implement Simple Linear Regression

Select one defensible numerical feature, such as living area for Ames, median income for California, or distance to transit for UCI. Explain why it was selected; do not choose it using test-set performance.

Simple Linear Regression

```
from sklearn.linear_model import LinearRegression


simple_feature = 'GrLivArea'  # replace for the dataset
simple_model = LinearRegression()
simple_model.fit(X_train[[simple_feature]], y_train)
simple_pred = simple_model.predict(X_test[[simple_feature]])


print('Intercept:', simple_model.intercept_)
print('Slope:', simple_model.coef_[0])
```

Observed values and fitted line

```
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(X_test[simple_feature], y_test, alpha=0.6, label='Observed')
order = np.argsort(X_test[simple_feature].to_numpy())
ax.plot(
   X_test[simple_feature].to_numpy()[order],
   simple_pred[order],
   linewidth=2,
   label='Fitted line'
)
ax.set_xlabel(simple_feature)
ax.set_ylabel(target)
ax.set_title('Simple Linear Regression')
ax.legend()
plt.show()
```

Step 9 - Implement Multiple Linear Regression

Multiple Linear Regression pipeline

```
linear_pipeline = Pipeline([
   ('preprocess', preprocess),
   ('model', LinearRegression())
])

linear_pipeline.fit(X_train, y_train)
linear_pred = linear_pipeline.predict(X_test)
```

Step 10 - Create a reusable evaluation function

Evaluation helper

```
def evaluate_regressor(name, fitted_model, X_eval, y_eval):
   pred = fitted_model.predict(X_eval)
   mae = mean_absolute_error(y_eval, pred)
   mse = mean_squared_error(y_eval, pred)
   rmse = np.sqrt(mse)
   r2 = r2_score(y_eval, pred)

   # Adjusted R^2 uses the transformed feature count.
   Xt = fitted_model.named_steps['preprocess'].transform(X_eval)
   n, p = Xt.shape
   adj_r2 = np.nan if n <= p + 1 else 1 - (1-r2)*(n-1)/(n-p-1)

   return {
      'Model': name,
      'MAE': mae,
      'MSE': mse,
      'RMSE': rmse,
      'R2': r2,
      'Adjusted_R2': adj_r2
   }, pred
```

Step 11 - Compare required regression models

Controlled model comparison

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

model_specs = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'Lasso': Lasso(alpha=0.001, max_iter=20000),
    'Elastic Net': ElasticNet(alpha=0.001, l1_ratio=0.5, max_iter=20000),
    'Decision Tree': DecisionTreeRegressor(max_depth=6, random_state=SEED),
    'Random Forest': RandomForestRegressor(
        n_estimators=300, min_samples_leaf=2,
        random_state=SEED, n_jobs=-1
    ),
    'Gradient Boosting': GradientBoostingRegressor(random_state=SEED)
}

fitted_models = {}
results = []
predictions = {}

for name, estimator in model_specs.items():
    pipe = Pipeline([('preprocess', preprocess), ('model', estimator)])
    pipe.fit(X_train, y_train)
    row, pred = evaluate_regressor(name, pipe, X_test, y_test)
    results.append(row)
    fitted_models[name] = pipe
    predictions[name] = pred

results_df = pd.DataFrame(results).sort_values('RMSE')
display(results_df)
```

Step 12 - Implement Polynomial Regression carefully

Use degree 2 on a small, justified set of numerical features. Applying polynomial expansion to dozens of variables can create an unnecessarily high-dimensional design matrix.

Degree-2 polynomial model with Ridge control

```
from sklearn.preprocessing import PolynomialFeatures

selected_numeric = ['GrLivArea', 'OverallQual', 'YearBuilt']  # Ames example
poly_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0))
])

poly_pipe.fit(X_train[selected_numeric], y_train)
poly_pred = poly_pipe.predict(X_test[selected_numeric])
```

```
from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {
    'mae': 'neg_mean_absolute_error',
    'rmse': 'neg_root_mean_squared_error',
    'r2': 'r2'
}

cv_rows = []
for name, estimator in model_specs.items():
    pipe = Pipeline([('preprocess', preprocess), ('model', estimator)])
    scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring=scoring, n_jobs=-1)
    cv_rows.append({
        'Model': name,
        'CV_MAE_mean': -scores['test_mae'].mean(),
        'CV_RMSE_mean': -scores['test_rmse'].mean(),
        'CV_RMSE_std': scores['test_rmse'].std(),
        'CV_R2_mean': scores['test_r2'].mean()
    })

cv_results = pd.DataFrame(cv_rows).sort_values('CV_RMSE_mean')
display(cv_results)
```

Step 13 - Use cross-validation for model comparison

Five-fold cross-validation on training data

Step 14 - Tune at least one model

Ridge hyperparameter tuning

```
from sklearn.model_selection import GridSearchCV

ridge_pipe = Pipeline([
    ('preprocess', preprocess),
    ('model', Ridge())
])

ridge_grid = {
    'model__alpha': np.logspace(-3, 3, 13)
}

search = GridSearchCV(
    ridge_pipe,
    param_grid=ridge_grid,
    scoring='neg_root_mean_squared_error',
    cv=cv,
    n_jobs=-1,
    return_train_score=True
)
search.fit(X_train, y_train)

print('Best parameters:', search.best_params_)
print('Best CV RMSE:', -search.best_score_)
best_ridge = search.best_estimator_
```

An alternative extension is RandomizedSearchCV for Random Forest or Gradient Boosting. The search space and computational budget must be stated. Do not select hyperparameters from the test set.

Step 15 - Diagnose overfitting and underfitting

Train-test performance gap

```
def split_metrics(model, X_train, y_train, X_test, y_test):
    rows = []
    for split_name, Xs, ys in [
        ('Train', X_train, y_train),
        ('Test', X_test, y_test)
    ]:
        pred = model.predict(Xs)
        rows.append({
            'Split': split_name,
            'MAE': mean_absolute_error(ys, pred),
            'RMSE': np.sqrt(mean_squared_error(ys, pred)),
            'R2': r2_score(ys, pred)
        })
    return pd.DataFrame(rows)

for name in ['Linear Regression', 'Decision Tree', 'Random Forest']:
    print(name)
    display(split_metrics(fitted_models[name], X_train, y_train, X_test, y_test))
```

Interpretation: high error on both train and test suggests underfitting; very low training error with substantially worse validation/test error suggests overfitting. Use cross-validation variability before making a final claim.

Step 16 - Perform residual analysis

Residual plots

```
selected_name = results_df.iloc[0]['Model']
y_pred = predictions[selected_name]
residuals = y_test.to_numpy() - y_pred

fig, ax = plt.subplots(figsize=(6.5, 5))
ax.scatter(y_pred, residuals, alpha=0.65)
ax.axhline(0, linestyle='--')
ax.set_xlabel('Predicted value')
ax.set_ylabel('Residual = actual - predicted')
ax.set_title(f'Residuals vs Fitted: {selected_name}')
plt.show()

fig, ax = plt.subplots(figsize=(6.5, 5))
sns.histplot(residuals, kde=True, ax=ax)
ax.set_title('Residual Distribution')
plt.show()
```

Actual versus predicted

```
fig, ax = plt.subplots(figsize=(6.5, 5))
ax.scatter(y_test, y_pred, alpha=0.65)
lo = min(y_test.min(), y_pred.min())
hi = max(y_test.max(), y_pred.max())
ax.plot([lo, hi], [lo, hi], linestyle='--')
ax.set_xlabel('Actual value')
ax.set_ylabel('Predicted value')
ax.set_title('Actual vs Predicted')
plt.show()
```

Step 17 - Inspect linear coefficients

Coefficient table

```
linear_fitted = fitted_models['Linear Regression']
feature_names = linear_fitted.named_steps['preprocess'].get_feature_names_out()
coefficients = linear_fitted.named_steps['model'].coef_

coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients,
    'AbsCoefficient': np.abs(coefficients)
}).sort_values('AbsCoefficient', ascending=False)

display(coef_df.head(20))
```

Interpret coefficients carefully

Coefficients depend on preprocessing and feature units. One-hot categories are interpreted relative to an encoded reference structure. Correlated predictors can make individual coefficients unstable even when prediction remains acceptable.

Step 18 - Optional target transformation

House prices are often right-skewed. A log transformation can reduce the influence of extreme values and convert multiplicative effects into an approximately additive form. Evaluate final errors after inverse transformation to original price units.

Log-target extension

```
from sklearn.compose import TransformedTargetRegressor

base_pipe = Pipeline([
   ('preprocess', preprocess),
   ('model', Ridge(alpha=1.0))
])

log_target_model = TransformedTargetRegressor(
   regressor=base_pipe,
   func=np.log1p,
   inverse_func=np.expm1
)
log_target_model.fit(X_train, y_train)
log_pred = log_target_model.predict(X_test)
```

Step 19 - Optional XGBoost comparison

Optional advanced boosting model

```
from xgboost import XGBRegressor

xgb_pipe = Pipeline([
   ('preprocess', preprocess),
   ('model', XGBRegressor(
      n_estimators=500,
      learning_rate=0.03,
      max_depth=4,
      subsample=0.8,
      colsample_bytree=0.8,
      objective='reg:squarederror',
      random_state=SEED,
      n_jobs=-1
   ))
])

xgb_pipe.fit(X_train, y_train)
```

XGBoost is optional. Students must not claim that it is superior without using the same validation protocol and reporting computational cost and interpretability trade-offs.

Step 20 - Save the selected pipeline and evidence

Reproducibility artifacts

```
import json
import joblib

final_model = best_ridge  # replace with the justified selected model
joblib.dump(final_model, 'house_price_pipeline.joblib')

results_df.to_csv('test_model_comparison.csv', index=False)
cv_results.to_csv('cross_validation_results.csv', index=False)

run_metadata = {
   'random_seed': SEED,
   'target': target,
   'train_rows': len(X_train),
   'test_rows': len(X_test),
   'selected_model': 'Tuned Ridge Regression'
}
with open('run_metadata.json', 'w') as f:
   json.dump(run_metadata, f, indent=2)
```

12. Models to Be Implemented

Model	Purpose in the study	Key settings to investigate	Requirement
Simple Linear Regression	One-feature visual and interpretive baseline	Feature choice; slope and intercept	Mandatory
Multiple Linear Regression	Main transparent multivariate baseline	Feature set; preprocessing; target transform	Mandatory
Polynomial Regression	Controlled nonlinear extension of linear modeling	Degree 2 or 3; selected features; regularization	Mandatory or instructor-approved substitute
Ridge Regression	Stabilize correlated coefficients and reduce variance	alpha / λ	Mandatory
Lasso Regression	Shrink and possibly zero coefficients	alpha; convergence; scaling	Mandatory
Elastic Net	Combine Ridge and Lasso behavior	alpha and l1_ratio	Recommended
Decision Tree Regressor	Nonlinear, interaction-capable single-tree benchmark	max_depth; min_samples_leaf	Mandatory
Random Forest Regressor	Variance-reduced nonlinear ensemble	n_estimators; depth; leaf size; max_features	Mandatory
Gradient Boosting / XGBoost	Sequential error-correcting ensemble	learning rate; estimators; depth; subsampling	Recommended extension

Minimum comparison set

The report must contain at least five meaningfully distinct models. Simple and multiple Linear Regression may be reported separately, but merely changing random seeds does not create a new model.

13. Evaluation Metrics and Diagnostics

13.1 Mean Absolute Error

$$\mathrm{MAE} = (1/n)\,\Sigma_{i=1}^{n}\,|y_i - \hat{y}_i| \quad \text{[MAE]}$$

MAE is in the target's original unit and answers: on average, how far is the prediction from the observed value? It is less sensitive to extreme errors than RMSE.

13.2 Mean Squared Error and Root Mean Squared Error

$$\mathrm{MSE} = (1/n)\,\Sigma_{i=1}^{n}\,(y_i - \hat{y}_i)^2 \quad \text{[MSE]}$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}} \quad \text{[RMSE]}$$

RMSE is in the original target unit and penalizes large errors strongly. It is useful when large valuation mistakes are especially costly.

13.3 Coefficient of determination

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2] \quad [R^2]$$

R^2 measures improvement over the mean baseline on the same data. It can be negative on unseen data. A high R^2 does not prove that assumptions are satisfied or that the model is unbiased across subgroups.

13.4 Adjusted R^2

$$\text{Adjusted } R^2 = 1 - (1 - R^2)(n - 1)/(n - p - 1) \quad [\text{Adjusted } R^2]$$

Adjusted R^2 penalizes the inclusion of additional predictors, where p is the effective number of features after preprocessing. It is mainly a descriptive complement and does not replace cross-validation.

13.5 Required evaluation protocol

Evidence	Purpose	Reporting requirement
Naive baseline	Establish whether learning adds value	MAE and RMSE
Training metrics	Identify fit capacity	MAE, RMSE and R^2
Cross-validation metrics	Estimate model-selection uncertainty	Mean and standard deviation
Final test metrics	Estimate generalization after selection	MAE, RMSE, R^2 and adjusted R^2
Residual diagnostics	Identify systematic model failure	Plots plus written interpretation
Segment-level errors	Identify unequal performance	At least two meaningful price or location segments

14. Visualization Requirements

Figure	Minimum content	Interpretation expected
1. Target distribution	Histogram/KDE; optionally log scale	Skewness, range and extreme values
2. Missing-value profile	Bar chart or table	Which variables need domain-aware treatment
3. Numerical correlation	Readable heatmap or ranked correlations	Relationships and multicollinearity clues
4. Feature-target relation	At least two numerical scatterplots	Linearity, nonlinearity and outliers
5. Category-price relation	Box/violin plot for at least one category	Location or quality effects and sample balance
6. Simple regression fit	Observed points and fitted line	Direction, strength and limitations
7. Actual vs predicted	Identity line included	Bias and spread across the target range
8. Residual vs fitted	Zero reference line	Nonlinearity, heteroscedasticity and outliers
9. Residual distribution / Q-Q	Histogram or Q-Q plot	Asymmetry and heavy tails
10. Model comparison	CV/test MAE or RMSE with labels	Accuracy-stability trade-off

Visualization quality

Every plot must have a title, axis labels, units where relevant, readable scale, and one or two sentences explaining the evidence. Decorative plots without analytical purpose receive no credit.

15. Experimentation Requirements

15.1 Minimum experiments

- Two datasets with clearly different characteristics.
- One simple Linear Regression model using a justified feature.
- One multiple Linear Regression model using a preprocessing pipeline.
- At least five total models, including a regularized model and a tree/ensemble model.
- Five-fold cross-validation or another justified validation strategy.
- Hyperparameter tuning for at least one model.
- Train-versus-validation/test analysis for underfitting and overfitting.
- Residual diagnostics for the linear baseline and final selected model.
- At least one ablation or controlled comparison, such as categorical features removed, target log transformation, or outlier policy.

15.2 Recommended experiment matrix

Experiment	Dataset	Model / change	Question answered
E0	Dataset A	Dummy mean baseline	Does any learned model add value?
E1	Dataset A	Simple Linear Regression	How informative is one major predictor?
E2	Dataset A	Multiple Linear Regression	What is gained from multiple attributes?
E3	Dataset A	Ridge / Lasso / Elastic Net	Does regularization improve stability?
E4	Dataset A	Decision Tree / Random Forest	Is nonlinearity important?
E5	Dataset A	Gradient Boosting or XGBoost	Does boosting improve predictive accuracy?
E6	Dataset B	Repeat common baseline set	Do findings generalize to a second dataset?
E7	Best dataset	Target transform or feature ablation	Which design choice causes improvement?

15.3 Model-selection rule

Choose the final model primarily using cross-validated training performance, variability, interpretability requirements, and computational cost. The test set is used once for final evidence. The smallest RMSE alone is not sufficient if the improvement is negligible, unstable, or operationally unjustified.

16. Result Tables and Templates

16.1 Dataset summary

Dataset	Rows	Raw predictors	Numeric	Categorical	Missing %	Target and unit	Coverage
Dataset A	___	___	___	___	___	___	___
Dataset B	___	___	___	___	___	___	___

16.2 Data-quality decisions

Issue	Evidence	Decision	Reason	Effect on row/feature count
Missing values	___	___	___	___
Duplicates	___	___	___	___
Outliers	___	___	___	___
High-cardinality category	___	___	___	___

16.3 Cross-validation comparison

Model	Key parameters	CV MAE mean	CV RMSE mean	CV RMSE SD	CV R^2 mean	Fit time
Linear Regression	___	___	___	___	___	___
Ridge	___	___	___	___	___	___
Lasso	___	___	___	___	___	___
Random Forest	___	___	___	___	___	___
Gradient Boosting	___	___	___	___	___	___

16.4 Final test comparison

Model	Train RMSE	Test MAE	Test RMSE	Test R^2	Adjusted R^2	Generalization note
___	___	___	___	___	___	___
___	___	___	___	___	___	___
___	___	___	___	___	___	___

16.5 Hyperparameter-tuning evidence

Model	Search method	Search space	Best setting	Best CV RMSE	Improvement over default
___	Grid / Random	___	___	___	___

16.6 Observation and interpretation table

Evidence	Observation	Interpretation	Action / limitation
Target distribution	___	___	___
Residual plot	___	___	___
Train-test gap	___	___	___
Segment errors	___	___	___
Coefficient / feature importance	___	___	___

17. Industry-Style Report Format

Recommended length: 8-12 pages excluding appendix, references, and full code. Use concise evidence, not notebook screenshots as the main report content.

Section	Required content	Suggested emphasis
1. Executive summary	Problem, data, selected model, headline metrics, major limitation	One page or less; readable by a manager
2. Business problem	Stakeholder, decision, target, error costs, intended and prohibited uses	Clarify prediction unit and currency
3. Dataset description	Source, coverage, schema, target, quality, limitations	Include summary table
4. Methodology	Split, preprocessing, models, validation, tuning and reproducibility	Explain leakage controls
5. EDA and feature engineering	Only findings that influence modeling	Connect plots to decisions
6. Model development	Baseline, comparative models, hyperparameters and experiment design	Maintain fair comparison
7. Evaluation results	CV and test metrics, diagnostics, segment errors	Use tables and well-labeled figures
8. Interpretation	Coefficients, feature influence, model behavior and trade-offs	Avoid causal language

Section	Required content	Suggested emphasis
9. Limitations and risk	Coverage, bias, drift, unusual properties, uncertainty, invalid uses	Be specific and evidence-based
10. Future improvements	New data, spatial/temporal split, uncertainty estimates, deployment monitoring	Prioritize actions
11. Conclusion	Answer whether the model meets the stated success criteria	No new results
Appendix	Code link, environment, additional plots and parameter tables	Keep main report concise

17.1 Executive-summary template

Template

This study developed and evaluated regression models for predicting [target and unit] using [dataset and coverage]. The final model, [model], achieved a test MAE of [value], RMSE of [value], and R^2 of [value], improving RMSE by [percentage] relative to the mean baseline. The strongest predictive signals were [features], while errors increased for [segment]. The model is suitable for [intended decision] within [coverage], but should not be used for [invalid use] without additional validation.

17.2 Model-recommendation template

Select [model] because it provides the best balance of cross-validated accuracy, stability, interpretability, and computational cost. Its improvement over [baseline] is [quantified]. The remaining risks are [specific residual pattern or coverage limitation]. Before deployment, the organization should [monitoring or data action].

18. Discussion Questions

1. Why is a mean-prediction baseline necessary even when Linear Regression is available?
2. What does a negative test R^2 imply?
3. Why may a feature have high correlation with price but a small multiple-regression coefficient?
4. How can neighborhood information improve accuracy while also creating governance concerns?
5. When would a temporal or grouped split be more realistic than a random split?
6. Why must imputation, scaling, and feature selection be fitted only on training folds?
7. What evidence distinguishes heteroscedasticity from random residual variation?
8. Why can Random Forest improve accuracy but weaken extrapolation and coefficient-style interpretation?
9. How does target log transformation change coefficient interpretation and error behavior?
10. What practical evidence is needed before using the model for mortgage or taxation decisions?
11. If two models have nearly identical RMSE, what other criteria should determine selection?
12. How would inflation and market drift affect a model trained on historical housing data?

19. Viva Questions

13. Define supervised learning and regression.
14. Distinguish simple from multiple Linear Regression.
15. What is a residual?
16. Why does OLS square the residuals?
17. State the interpretation of a regression coefficient.
18. What is data leakage? Give one example from this lab.
19. Why is the test set kept untouched during tuning?
20. Compare MAE and RMSE.
21. Can R^2 be negative? Explain.
22. Why is scaling important for Ridge and Lasso?
23. How do Ridge and Lasso differ?
24. Why might Lasso set some coefficients to zero?
25. Is Polynomial Regression a linear model? Explain.

26. What is multicollinearity and how can it affect coefficients?
27. What is heteroscedasticity?
28. How do you detect overfitting from train and test metrics?
29. What does cross-validation estimate?
30. Why can a Decision Tree fit nonlinear relationships?
31. How does Random Forest reduce variance?
32. Why should model errors be analyzed by price segment?
33. What is the role of a pipeline in scikit-learn?
34. What is the difference between prediction and causal inference?
35. Why can a model fail when moved to a different city or year?
36. What information should accompany a saved model artifact?

20. Common Mistakes to Avoid

Mistake	Why it is wrong	Correct practice
Using the test set repeatedly	Transforms the test set into a validation set and biases performance	Select using cross-validation; evaluate once on test
Preprocessing before splitting	Leaks distributional information into training	Fit preprocessing inside a pipeline on training folds
Dropping all missing rows	May discard information and bias the sample	Use domain-aware imputation and quantify impact
Treating IDs as predictors	Can memorize observations without generalizable meaning	Remove or justify grouping use
Using “accuracy” for regression	Classification accuracy is not defined for continuous targets	Use MAE, RMSE, R^2 and diagnostics
Selecting the model only by training error	Rewards overfitting	Use cross-validation and held-out evaluation
Interpreting association as causation	Observational regression does not establish interventions	Use conditional association language
Reporting only R^2	Hides error magnitude and segment behavior	Report MAE, RMSE and residual evidence
Ignoring units	Makes business interpretation impossible	State currency, scale and transformed units
Uncontrolled outlier deletion	Can artificially improve results and remove valid rare homes	Use documented rules and sensitivity analysis
Inconsistent preprocessing across models	Creates unfair comparison	Use comparable pipelines or justify differences
Submitting unexecuted code	Outputs may not correspond to current code	Restart kernel and run all before submission

21. Submission Guidelines

21.1 File naming

Recommended naming convention

```
REGNO_Lab01_HousePrice.ipynb
REGNO_Lab01_HousePrice_Report.pdf
REGNO_Lab01_README.md
REGNO_Lab01_Results.csv
REGNO_Lab01_Model.joblib   # optional
```

21.2 Notebook requirements

- Runs from beginning to end after a fresh kernel restart.
- Contains relative paths or clear data-loading instructions.
- Uses fixed seeds and reports package versions.
- Separates markdown explanation from code.
- Contains no hidden manual edits to result tables.

- Includes final summary and limitations in the notebook itself.

21.3 Report requirements

- Use original analysis and original written interpretation.
- Number all figures and tables and refer to them in the text.
- Cite dataset and external code or documentation sources.
- Place long code listings in the appendix or repository, not the main report.
- State team-member contributions when group work is permitted.

22. Assessment Rubric

Criterion	Evidence expected	Marks
Problem framing and success criteria	Correct target, user, decision context, error costs and scope	8
Dataset understanding and documentation	Source, schema, units, coverage, limitations and data dictionary	10
Data quality and preprocessing	Missing values, outliers, encoding, scaling, leakage-safe pipeline	14
EDA and analytical reasoning	Relevant plots with defensible observations connected to decisions	10
Linear Regression theory and implementation	Simple and multiple models, equations, coefficient interpretation	14
Comparative modeling and tuning	At least five models, cross-validation and justified hyperparameter search	14
Evaluation and diagnostics	Baselines, metrics, residuals, assumptions, train-test gap and segment errors	14
Interpretation and recommendation	Model choice, trade-offs, limitations, business meaning and future work	8
Reproducibility and code quality	Executable notebook, modular code, versions, seed and artifacts	4
Report quality and academic integrity	Professional structure, citations, clarity and originality	4

Total: 100 marks

22.1 Performance descriptors

Level	Descriptor
Excellent (85-100)	Reproducible, analytically rigorous, fair model comparison, strong diagnostics, precise interpretation and credible limitations.
Good (70-84)	Correct end-to-end workflow with minor omissions in diagnostics, justification or presentation.
Satisfactory (55-69)	Core model works, but comparison, reasoning, leakage control, or interpretation is limited.
Needs improvement (<55)	Incomplete or non-reproducible workflow, incorrect evaluation, substantial leakage, or unsupported conclusions.

23. Responsible Analytics and Academic Integrity

23.1 Responsible use

- Housing prices can encode historical socioeconomic and spatial inequalities. Location features and proxies require governance when models influence credit, taxation, or access.
- Do not use the laboratory model as a production valuation system without current local data, uncertainty estimation, human review, and monitoring.
- Report where the data are sparse and where predictions require extrapolation.
- Avoid presenting feature importance as proof of causal impact.

23.2 Academic integrity

- External code may be consulted only when cited and understood.

- Students must be able to explain every submitted code block and modeling decision.
- Fabricated metrics, manually altered plots, or copied interpretation constitute academic misconduct.
- Generative tools, where permitted by course policy, must be disclosed and must not replace independent reasoning or verification.

24. Final Submission Checklist

☐ Problem, target, observation unit, and intended use are clearly stated.

☐ Dataset source, target unit, coverage, and limitations are documented.

☐ Raw data remain unchanged; cleaning decisions are reproducible.

☐ Train-test split occurs before learned preprocessing.

☐ A pipeline handles missing values, categorical encoding, and scaling.

☐ Naive, simple-linear, and multiple-linear baselines are reported.

☐ At least five models are compared with the same validation protocol.

☐ At least one model is tuned using cross-validation.

☐ MAE, RMSE, R^2, adjusted R^2, and residual diagnostics are included.

☐ Train-test or train-validation gaps are discussed.

☐ At least two datasets are used in the extended study.

☐ Figures have titles, labels, units, and written interpretations.

☐ Final model selection is justified beyond a single metric.

☐ Limitations, responsible-use concerns, and future improvements are specific.

☐ Notebook restarts and runs completely; all files follow the naming convention.

25. References and Dataset Sources

Students should cite the exact dataset version they use. The following sources support this manual and provide authoritative dataset descriptions.

1. UCI Machine Learning Repository. Real Estate Valuation dataset (ID 477). https://archive.ics.uci.edu/dataset/477/real+estate+valuation+data+set
2. scikit-learn documentation. California Housing dataset and fetch_california_housing. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
3. scikit-learn User Guide. Real-world datasets: California Housing. https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset
4. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. Journal of Statistics Education, 19(3). https://jse.amstat.org/v19n3/decock.pdf
5. Kaggle. House Prices - Advanced Regression Techniques. https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques
6. James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. An Introduction to Statistical Learning with Applications in Python.
7. Kuhn, M. and Johnson, K. Applied Predictive Modeling; and Feature Engineering and Selection.
8. Hastie, T., Tibshirani, R., and Friedman, J. The Elements of Statistical Learning.
9. scikit-learn User Guide. Model selection, preprocessing, pipelines, linear models, ensembles, and metrics. https://scikit-learn.org/stable/user_guide.html

Instructor Adaptation Note

The instructor may designate one dataset for the 3-hour core lab and reserve the remaining datasets and advanced models for a graded extension. For an introductory cohort, use UCI or California Housing in the laboratory and Ames for the extended report. For a stronger cohort, use Ames throughout, but provide the data dictionary and a constrained feature set for the core session.

Recommended final design

Core laboratory: UCI or California Housing with simple and multiple Linear Regression, preprocessing, metrics and residuals. Extended study: Ames Housing with at least five models, cross-validation, tuning, diagnostics and an industry-style report.